

# Uber

## Functional requirements:

- user should be able book cab based on his preference,
- user should be able to search for locations based on his preference,
- rider should be able to accept/reject any ride based on his preference,

## Non-functional requirements:

- Data-partitioning,
- load balancing,
- fault tolerance,
- vertical scalability,
- horizontal scalability,
- low latency,
- high throughput,

CAP theorem (Here we need to prioritise Strong consistency in order to avoid double booking/double payment, & for search scenario we need to prioritise 99.99% uptime i.e Availability>>Consistency)

## Core Entities:

- user ,
- book,
- payment

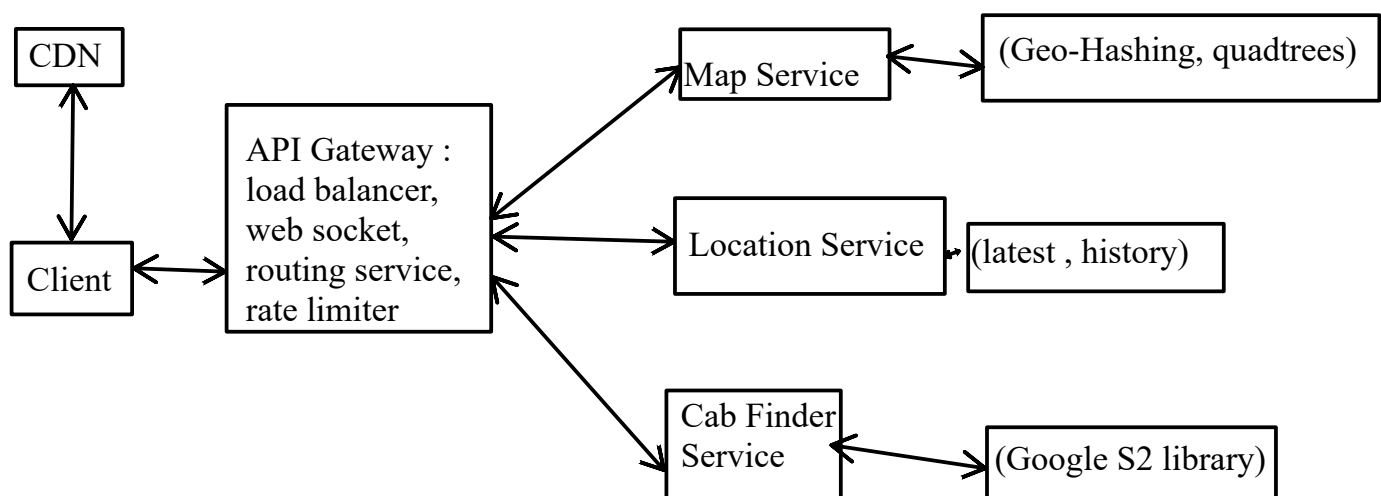
## API :

POST:/booking

```
{  
  userID,  
  bookingStatus  
}
```

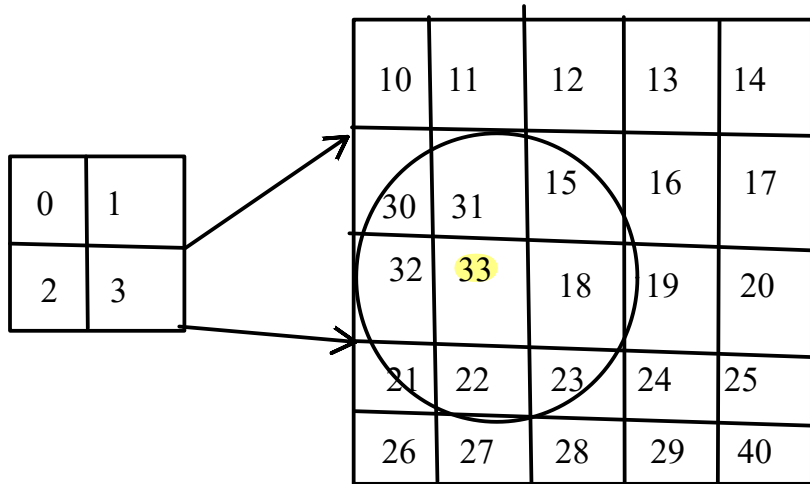
GET:/search/{locationID}

## HLD:

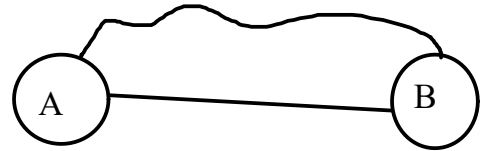


## GeoHashing

(lat,long) -> chunkID



## Google S2 library



We cannot use  
Euclidian distance computation  
because path between any two points  
will not be straight line  
so only we are supposed to use google  
s2 library

Consistent hashing ,  
Gossip protocol